

Real-Time Scenario: Querying Specific Columns from Parquet in S3 without Athena

Project Title

Querying a Specific Column from a Parquet File in Amazon S3 using S3 Select (No Athena)

Problem Scenario

A data engineer at CloudInsights Inc. needed to extract a single column from a Parquet file stored in an Amazon S3 bucket. The challenge was to perform this operation without using higher-level query services like Amazon Athena, EMR, or Redshift, aiming instead for **S3 Select** due to its minimal setup and low cost.

However, the attempt to query the Parquet file using S3 Select failed with the following error:

"The specified method is not allowed against this resource."

Root Cause:

S3 Select supports only **flat Parquet files** with compatible encoding. The original file was **not S3 Select-compatible**, likely due to nested schemas or unsupported encoding.

Solution Overview

Convert to a Flat Parquet File Format

To make the file compatible with S3 Select:

1. **Re-exported** the Parquet file with a flat schema using Python libraries.
 2. **Uploaded** the clean Parquet file back to S3.
 3. **Used S3 Select** to query specific columns directly in the S3 Console.
-

Tools & Services Used

- Amazon S3
 - S3 Select
 - Python (pandas + pyarrow)
 - (Optional) AWS CLI for local testing
-

Folder/File Details

File Name	Description
<code>all.parquet</code>	Original Parquet file with complex or unsupported schema
<code>flat.parquet</code>	Re-exported flat Parquet file compatible with S3 Select
<code>titanic.csv</code>	Source CSV file (optional) used to generate <code>flat.parquet</code>

Step-by-Step Implementation

Upload Original File

• Uploaded `all.parquet` to: `s3://nyc-taxi-dataset-bucket/all.parquet`

Attempt Query with S3 Select (Failed)

Query Attempted:

```
SELECT s.Survived FROM s3object s;
```

Error:

"The specified method is not allowed against this resource."

Convert CSV to Flat Parquet Format

Python Code:

```
import pandas as pd

# Load raw CSV
df = pd.read_csv("titanic.csv")

# Convert to S3 Select-compatible Parquet
df.to_parquet("flat.parquet", engine="pyarrow", index=False)
```

Upload `flat.parquet` to S3

Location:

```
s3://nyc-taxi-dataset-bucket/flat.parquet
```

Run S3 Select Query (Successful)

1. Go to S3 console
2. Click `flat.parquet`
3. Choose **"Query with S3 Select"**
4. Format: `Parquet`
5. Compression: `None`
6. Run SQL query:

```
SELECT s.Survived FROM s3object s;
```

To preview full data:

```
SELECT * FROM s3object s LIMIT 10;
```

IAM Permissions

Ensure the IAM role or user has the following permissions:

```
{
  "Effect": "Allow",
  "Action": [
    "s3:GetObject",
    "s3:ListBucket"
  ],
  "Resource": [
    "arn:aws:s3:::nyc-taxi-dataset-bucket",
    "arn:aws:s3:::nyc-taxi-dataset-bucket/*"
  ]
}
```

Key Learnings

- S3 Select is **fast and cost-effective** for column-level queries.
- It only supports **flat, simple Parquet files**.
- You can fix unsupported files using `** + **` by re-exporting them.

- You can perform data exploration without setting up Athena, EMR, or Redshift.

Project Status: Completed

- S3 Select worked successfully on the re-exported Parquet file.
- No external query engines used — minimal AWS service dependency.

Workspace Output (Paste Your Results Below)

 Paste screenshots, query results, error messages, and log snippets:

```
[S3 Screenshot showing flat.parquet file]
[S3 Select query result output for SELECT * query]
[Console screenshot showing successful column-level query]
[Error message from original unsupported file query]
```

Summary

This project demonstrates how to:

- Diagnose and fix Parquet compatibility issues with S3 Select
- Use minimal tools to extract insights directly from S3
- Implement a low-cost, serverless querying mechanism

Useful in scenarios with limited permissions or infrastructure!

 Prepared by: Atmakuru Siva Sandeep\  17 Date: 23 June 2025